

Práctica Deep Learning – Predicción de Engagement en POIs Turísticos

Autora: Leticia Cabañas Morales

Keepcoding- Junio 2025

Índice

1. Objetivo
2. Descripción del Dataset
3. Metodología
 - 3.1 Reproducibilidad y configuración inicial
 - 3.2 Preparación y Análisis de Datos
 - 3.3 Arquitectura del Modelo
 - 3.4 Entrenamiento y Optimización
 - 3.5 Evaluación y Análisis del Modelo
4. Interpretabilidad del Modelo
5. Conclusiones y Propuestas Futuras

1. Objetivo

El propósito de este proyecto es desarrollar un modelo avanzado de Deep Learning capaz de predecir con precisión el nivel de *engagement* que generarán distintos puntos de interés turísticos (POIs). El modelo integra dos tipos de información complementaria:

- **Características visuales** extraídas mediante redes neuronales convolucionales (CNN), que analizan elementos visuales distintivos en las imágenes de los POIs.
- **Metadatos estructurados**, como coordenadas geográficas, categorías, etiquetas, popularidad y otros atributos contextuales.

El enfoque multimodal empleado permite capturar tanto la reacción estética del usuario como factores prácticos, lo que mejora la capacidad predictiva en comparación con modelos unidimensionales

Entre los objetivos específicos destacan:

- Optimizar la selección de contenido para maximizar la interacción del usuario.
- Identificar patrones visuales asociados a mayor *engagement*.
- Mejorar la experiencia del usuario destacando contenido relevante.
- Generar recomendaciones y *insights* basados en datos para la toma de decisiones.

2. Descripción del Dataset

El conjunto de datos ha sido recopilado desde la plataforma *Artgonuts* e incluye información detallada sobre **1.569 POIs**, estructurada en:

Imágenes

Cada POI dispone de una imagen principal representativa, cuya ruta se encuentra en la columna `main_image_path`.

Metadatos

id: Identificador único del POI.

name: Nombre del punto de interés.

shortDescription: Breve descripción textual.

locationLat, locationLon: Coordenadas geográficas.

barrio, distrito: Información geográfica adicional.

categories: Temática o tipo de atracción.

tier: Nivel de popularidad o relevancia.**tags:** Etiquetas descriptivas.

xps: Métrica de gamificación (puntos de experiencia).

Métricas de Engagement

Visits: Número de visitas registradas.

Likes, Dislikes: Valoraciones positivas y negativas.

Bookmarks: Número de veces que el POI ha sido guardado como favorito.

El diseño del dataset permite flexibilidad en la selección de variables, fomentando la justificación analítica y el enfoque en aquellas más relevantes para el modelo predictivo. El dataset se encuentra organizado jerárquicamente, con imágenes almacenadas en carpetas por ID y archivos de metadatos estructurados asociados.

3. Metodología

3.1 Reproducibilidad y configuración inicial

Para garantizar la reproducibilidad de los experimentos, se establecieron semillas aleatorias para los distintos entornos (**random**, **numpy** y **torch**), además de fijar los parámetros de CUDNN para obtener resultados deterministas. También se comprobó automáticamente la disponibilidad de GPU para aprovechar la aceleración por hardware durante el entrenamiento del modelo.

Se han instalado las dependencias con **requirements.txt**. Se han importado las librerías clave, y se han cargado desde **module_utils.py** las funciones auxiliares utilizadas en la práctica.

3.2 Preparación y Análisis de Datos

Se cargó el dataset **poi_dataset.csv**, se eliminaron variables irrelevantes para el modelo, o que requerían tratamiento NLP avanzado (id, name, shortDescription) y se analizaron estadísticamente las variables numéricas, identificando distribuciones sesgadas y escalas heterogéneas. Por este motivo, se aplicaron transformaciones logarítmicas a las métricas de engagement (Visits, Likes, Dislikes y Bookmarks) antes de definir la variable objetivo, y se normalizaron las variables numéricas.

Análisis y preprocesamiento de variables categóricas:

Las columnas categories y tags contenían listas representadas como cadenas. Se transformaron a listas reales para su posterior procesamiento. El análisis mostró:

- **categories:** La mayoría de los POIs tiene exactamente 3 categorías. Se aprecia un desbalanceo evidente.
- **tags:** Hay un número muy elevado de etiquetas diferentes. Presentan una distribución bastante desbalanceada. La gran mayoría de los POIs tienen varios tags, especialmente 10 y 13. Se observaron algunas listas vacías en ambas variables, pero no fueron tratadas de ninguna manera.

Dado que muchos POIs presentan múltiples categorías, se optó por una representación mediante **multi-hot encoding**, generando vectores binarios por muestra donde cada posición indica la presencia o ausencia de una categoría o etiqueta específica. Esta

codificación resulta especialmente útil en tareas de clasificación donde la semántica contextual juega un papel importante.

En el caso de los tags, debido a su alta cardinalidad y distribución desbalanceada, se seleccionaron únicamente los 10 más frecuentes para su representación individual. El resto de etiquetas se agruparon en una columna adicional `tag_other`. Esta estrategia permite capturar las etiquetas más representativas sin incrementar excesivamente la dimensionalidad del modelo, mejorando así la eficiencia y capacidad de generalización.

Distribución y correlaciones de variables numéricas:

La variable **tier** se decidió utilizar como una variable ordinal, ya que sus valores reflejan distintos niveles de popularidad. Se verificó que el valor 1 se asociaba con un mayor número de Likes, indicando una mayor relevancia, mientras que el valor 4 correspondía a los POIs menos populares.

La variable **xps**, aunque es discreta, tiene un valor cuantitativo que el modelo puede explotar directamente, por lo que se decidió tratarla como numérica.

Se evaluaron la distribución y la relación entre las métricas de engagement: Visits, Likes, Dislikes y Bookmarks. Se observó lo siguiente:

- **Likes y Bookmarks** están fuertemente correlacionadas, lo cual era de esperar. No obstante, dado que los Bookmarks son mucho menos frecuentes, se interpretan como acciones que implican una mayor intención.
- **Dislikes** muestra una correlación negativa moderada con Likes y Bookmarks, por lo que se decidió penalizar su presencia en la métrica de engagement.
- **Visits** prácticamente no presenta correlación con ninguna de las otras métricas, lo que sugiere que el número de visitas no está directamente relacionado con que el POI guste o se guarde.

Se aplicó una transformación logarítmica a estas variables para reducir su asimetría. A continuación, se definió una métrica compuesta llamada **engagement_score**, en la que se asignó mayor peso a los Bookmarks (por ser una acción más comprometida) y se penalizó con fuerza la presencia de Dislikes, además de considerar Likes y Visits.

Dado que la distribución de **engagement_score** era multimodal, se planteó el problema como una clasificación multiclase (**engagement_level**), dividiendo el score en tres niveles:

- Bajo (0)
- Medio (1)
- Alto (2)

La división se realizó utilizando los percentiles Q1 y Q3, lo que facilita la interpretación y mejora la capacidad del modelo para aprender. Como se detectó cierto desbalance entre las clases, se optó por realizar una división estratificada de los datos

Preprocesamiento de Imágenes y Metadatos

Las imágenes se redimensionaron a 224x224 y normalizaron con medias y desviaciones estándar de ImageNet. Para entrenamiento se aplicó **data augmentation** (rotación aleatoria, alteración del color), con el objetivo de mejorar la robustez y capacidad de generalización del modelo.

Los metadatos numéricos (xps, locationLat, locationLon) se estandarizaron con **StandardScaler** para facilitar el aprendizaje del modelo y prevenir que las variables con escalas mayores dominen durante el entrenamiento.

División de los datos y creación de DataLoaders

Se realizó una **división estratificada** en tres conjuntos de datos:

- Entrenamiento: 70%
- Validación: 15%
- Test: 15%

Se definió una clase personalizada llamada POIDataset para integrar de forma conjunta tanto las imágenes como los metadatos en un único flujo de entrenamiento. Su correcto funcionamiento fue validado mediante ejemplos individuales.

Posteriormente, se crearon los DataLoaders correspondientes para cada conjunto, utilizando un batch size de 256.

3.3 Arquitectura del Modelo

El modelo final está compuesto por dos ramas principales:

- **Rama visual:** Se utilizó una CNN preentrenada (**ResNet18**), eliminando su capa final de clasificación. Esta red permite extraer representaciones visuales profundas de las imágenes sin necesidad de entrenar desde cero.
- **Rama de metadatos:** Para procesar los metadatos estructurados, se definió un bloque **MLP** (Multilayer Perceptron) compuesto por dos capas *fully connected*, activación ReLU, normalización BatchNorm1d para estabilizar y acelerar el entrenamiento, y Dropout para evitar el sobreajuste.

Las salidas de ambas ramas se concatenan y se pasan por un bloque adicional de clasificación (**fusion_branch**), compuesto por otra capa oculta con normalización y dropout, finalizando en una capa de salida de 3 unidades, correspondiente a las clases del engagement.

Prevención de *data leakage*

Se tuvo especial cuidado en evitar fuga de información entre los conjuntos de entrenamiento, validación y prueba. Las transformaciones aplicadas a los metadatos (como el escalado) se ajustaron exclusivamente sobre el conjunto de entrenamiento, y posteriormente se aplicaron a los conjuntos de validación y prueba.

El modelo fue probado utilizando un *batch* del conjunto de entrenamiento para verificar la correcta integración de las entradas y el funcionamiento de la red. La salida fue una predicción en forma de tensor de 3 clases, con un *batch size* de 256, como se esperaba.

3.4 Entrenamiento y Optimización

Se implementó un modelo **multimodal** compuesto por:

- **Rama visual:** ResNet-18 preentrenada en ImageNet. Se eliminó la capa fully-connected final y se usaron sus embeddings (512-D).
- **Rama tabular:** MLP con dos capas (BatchNorm + activación ELU + Dropout).
- **Fusión:** concatenación [features_imagen, features_tabulares] seguida de un MLP de clasificación a 3 clases.

Para lidiar con el **desbalanceo** de `engagement_level`, se empleó **CrossEntropyLoss** con **class_weights** calculados en `y_train`. Además, se aplicó **label smoothing** para reducir sobreajuste y hacer el clasificador menos “sobreconfidente”.

Optimizador y regularización:

- **AdamW** (mejor manejo de `weight_decay` que Adam).
- **ReduceLROnPlateau** monitorizando `val_acc` (reduce el LR cuando la validación se estanca).
- **Early Stopping** con paciencia de 5–6 épocas.
- **Data augmentation** moderado en imágenes (rotación, jitter de color, crop aleatorio) y normalización ImageNet.

Se siguieron tres fases, todas con **división estratificada** y **semillas fijadas** para reproducibilidad:

1. Baseline (backbone congelado)

- LR (cabeza): $1e-3$, `weight_decay`= $3e-5$, `label_smoothing`=0.05.
- Resultado validación: **~86.4 %**.

2. Fine-tuning parcial del backbone

- LR discriminativo: cabeza $5e-4$, `weight_decay`= $3e-5$, backbone $5e-5$.
- Resultado validación: **~90.9 %**.
- Observación: ligera mejora global.

3. Optimización con Optuna (train/val)

Espacio de búsqueda acotado alrededor de los mejores valores empíricos:

- $\text{lr_head} \in [2\text{e-}4, 1\text{e-}3]$ (log), $\text{lr_backbone} \in [1\text{e-}5, 2\text{e-}4]$ (log)
- $\text{weight_decay} \in [1\text{e-}6, 3\text{e-}4]$ (log)
- $\text{dropout} \in [0.30, 0.60]$, $\text{label_smoothing} \in [0.03, 0.12]$
- Pruner: **MedianPruner**, 40 trials.
- **Mejores hiperparámetros** encontrados:
 $\text{lr_head}=6.8\text{e-}4$, $\text{lr_backbone}=2.4\text{e-}5$, $\text{weight_decay}\approx 3.0\text{e-}6$,
 $\text{dropout}\approx 0.35$, $\text{label_smoothing}\approx 0.11$.
- **Mejor validación: 93.43 %** (epoch 11).
- Observación: Optuna mejora resultado de Val_Acc frente a fine-tuning manual. El scheduler redujo LR con buen “timing” y el early stopping evitó sobreajuste.

Fase	Hiperparámetros	Val Accuracy	Test Accuracy	
Baseline	$\text{lr}=1\text{e-}3$, $\text{dropout}=0.4$	86.4%	---	
Fine-tuning	$\text{lr_head}=5\text{e-}4$, $\text{lr_backbone}=5\text{e-}5$	90.9%	---	
Optuna	$\text{lr_head}=6.8\text{e-}4$, $\text{lr_backbone}=2.4\text{e-}5$, $\text{dropout}=0.35$	89.9%	88.8%	

Optuna permitió encontrar la mejor combinación de hiperparámetros, alcanzando 89.9% de accuracy en validación al entrenar nuestro modelo con dichos hiperparámetros.

3.5 Evaluación y Análisis del Modelo

Con el mejor estado del modelo (seleccionado en validación), la evaluación en el conjunto de **test** arrojó:

- Accuracy: **88.84 %**
- Macro-F1: **0.875**
- Informe por clase:

	Clase	Precisión	Recall	F1	Soporte
0		0.9907	0.8770	0.9304	122
1		0.8065	0.8621	0.8333	58
2		0.7937	0.9434	0.8621	53

Lectura de los resultados:

- **Clase 1** (antes problemática) mejora sensiblemente su **recall** (0.86) manteniendo una precisión alta (0.81).
- **Clase 2** prioriza **recall** (0.94) a costa de algo de precisión (0.79), lo que reduce falsos negativos (preferible si quieres captar “alto engagement”).
- **Clase 0** se mantiene fuerte y estable.

Análisis de errores:

- Las principales confusiones se concentran entre **clases 1 y 2** (líneas 2–3 de la matriz).
- Tras Optuna, se reducen los **falsos negativos** de la clase 2, a cambio de algunos **falsos positivos** hacia 2 desde 1.
- Este patrón es coherente con el objetivo de priorizar detección de **alto engagement** (clase 2) por encima del riesgo de “sobre-asignación” en casos limítrofes.

4. Interpretabilidad del Modelo

Para interpretar la rama visual se aplicó **Grad-CAM** sobre la última capa convolucional de ResNet-18. Se muestran mosaicos de ejemplos con el mapa de activación superpuesto a la imagen original (se incluyen aciertos y errores).

Hallazgos cualitativos:

- En predicciones correctas de **clase 2**, la activación se concentra en regiones visuales “salientes” (p. ej., patrones estéticos distintivos), lo que sugiere atención a rasgos informativos.
- En algunos errores 1–2, el mapa se desplaza a zonas poco discriminativas o es difuso, alineándose con las confusiones observadas.
- En **clase 0**, las activaciones tienden a ser más dispersas (consistente con la ausencia de señales visuales fuertes).

5. Conclusiones y Propuestas Futuro

El modelo multimodal entrenado logra una mejora sustancial frente a enfoques unidimensionales, alcanzando un 93.4% en validación y 88.8% en test.

Claves técnicas que funcionaron:

- **LR discriminativo** (cabeza > backbone) y **AdamW** con ReduceLROnPlateau.
- **Label smoothing** (~0.03–0.05) para recortar gap train-val.
- **Optuna** para afinar LR/weight_decay/dropou

Grad-CAM mostró que la red visual aprende patrones relevantes.

Mejoras potenciales:

- **Augmentación específica** para la clase 1 (p. ej., RandomResizedCrop más agresivo y ligeros cambios de color).
- **Freeze/Unfreeze progresivo** (One-Cycle LR o cosine annealing) para exprimir el backbone.
- **Análisis de datos:** revisar outliers de clase 1 mal clasificados con Grad-CAM, y considerar técnicas de **reweighting focal** si interesa subir aún más recall minoritario.